

Projektvorgehen

Modellierung und Simulation eines Photovoltaikmoduls

VU Kontinuierliche Simulation, 325.118 – Arbeitsgrundlage für die Gruppe

Contents

1	Was verlangt wird	1
2	Das Modell in Kurzform	2
3	Rollen	2
4	Reihenfolge und Meilensteine	3
5	Ordnerstruktur	4
6	Arbeitsregeln für den Code	4
6.1	Zwei technische Stolpersteine	5
7	Arbeitsregeln für das Protokoll	5
7.1	Die Annahmentabelle	5
7.2	Abbildungen	6
7.3	KI-Werkzeuge	6
8	Offene Punkte	6
9	Woran wir merken, dass es gut läuft	7

Zweck dieses Dokuments. Es legt fest, wer was macht, in welcher Reihenfolge gearbeitet wird und nach welchen Regeln Code und Protokoll entstehen. Wer eine Regel brechen will, darf das, muss es aber ansagen und im Entscheidungslog vermerken. Das Ziel ist, dass am Ende jeder von uns jeden Teil erklären kann, auch den, den er nicht selbst geschrieben hat.

1 Was verlangt wird

Die Angabe nennt acht Punkte:

1. Modellbildung
2. Implementierung der Systemgleichungen in MATLAB
3. Literaturrecherche der Modellparameter
4. Validierung des Modells
5. Anwendungsfall
6. Sensitivitätsanalyse

7. Implementierung in Simulink

8. Protokoll und Präsentation

Entscheidend ist der Satz am Ende der Angabe: jeder Schritt muss begründbar und nachvollziehbar sein. Bewertet wird also nicht in erster Linie, ob die Kurve stimmt, sondern ob wir sagen können, warum wir sie so gerechnet haben. Das Protokoll ist entsprechend gewichtet: das „Warum“ steht über dem „Was“.

2 Das Modell in Kurzform

Bilanzraum ist das Modul ohne Rahmen, betrachtet als ein einziger Temperaturknoten. Aus dem ersten Hauptsatz folgt

$$C_m \frac{dT_m}{dt} = Q_{\text{solar}} - W_{\text{el}} - Q_{\text{konv}} - Q_{\text{rad}}. \quad (1)$$

Das Modul hat genau einen unabhängigen Energiespeicher, seine Wärmekapazität. Der Zustandsvektor hat damit die Ordnung eins, und zwar nicht weil wir das so gewählt haben, sondern weil die Physik es vorgibt. Dieser Satz gehört in die Präsentation, er ist der sauberste Bezug zur Vorlesung.

Die elektrische Leistung hängt selbst von T_m ab,

$$W_{\text{el}} = G A (\tau \alpha) \eta_{\text{ref}} [1 - \beta_{\text{ref}}(T_m - T_{\text{ref}})]. \quad (2)$$

Genau diese Rückkopplung macht aus der Bilanz eine Differentialgleichung. Ohne sie wäre T_m eine explizite Formel und das ganze Projekt wäre ein Taschenrechnerproblem.

Was wir bewusst weglassen

Wir rechnen das **Basismodell**. Konvektion über einen linearen Ansatz $h = h_a + h_b v$, nicht über die vier Nusselt-Strömungsregime von Tuncel et al. mit Windrichtung. Gründe:

- Die Windrichtung aus den GeoSphere-Daten lässt sich ohne Kenntnis der genauen Modulausrichtung nicht sinnvoll nutzen.
- Wir können die vier Regime mit unseren Daten nicht validieren, wir würden also Komplexität hinzufügen, die wir nicht prüfen können.
- Die Angabe verlangt ausdrücklich „sinnvolle Annahmen und Vereinfachungen“. Weglassen mit Begründung ist die Aufgabe, nicht deren Umgehung.

Wer den Nusselt-Ansatz später doch nachrüsten will: es ist genau eine Datei zu ändern, `calc_h_conv.m`. Der Rest des Codes bleibt unberührt. So ist die Struktur gebaut.

3 Rollen

Rolle	Zuständig für
A – Parameter	Literaturrecherche aller Modellparameter, Füllen von <code>init_parameters.m</code> , Parametertabelle im Protokoll, Herleitung von C_m aus dem Schichtaufbau.
B – Modellkern	MATLAB-Kern (ODE-Funktion, Solverwahl, Wetterinterpolation), Validierung gegen die Paperdaten, Kapitel Numerik und Validierung.

Rolle	Zuständig für
C – Daten und Anwendung	GeoSphere-Daten beschaffen und aufbereiten, Anwendungsfall, Sensitivitätsanalyse, Kapitel 6 und 7.
D – Simulink	Simulink-Modell, Kopplung mit dem Batteriemodell, Vergleich gegen die MATLAB-Lösung, Kapitel 8.
Alle	Modellbildung in der Startsituation, Annahmentabelle, Präsentation, Gegenlesen der Kapitel der anderen.

Der Modellkern wird von genau einer Person geschrieben. Vier Leute an derselben ODE-Funktion ist der schnellste Weg zu Code, den hinterher niemand mehr versteht. Wer etwas am Kern braucht, sagt es Person B.

Eine Person hat zusätzlich das Schlusswort über die Struktur des Protokolls. Das ist keine inhaltliche Autorität, sondern verhindert nur, dass am Ende zehn Kapitel in zehn verschiedenen Stilen dastehen.

4 Reihenfolge und Meilensteine

Phase	Inhalt	Wer
0	Papers lesen, Bilanzgleichung gemeinsam auf Papier, Systemgrenze festlegen, erste Annahmenliste	alle
0	GeoSphere-Daten herunterladen (24.06.–01.07.2019, 10-Minuten-Werte)	C
1	Parameterrecherche, <code>init_parameters.m</code> füllen	A
1	MATLAB-Kern lauffähig, Testlauf mit Platzhalterdaten	B
2	Paperdaten digitalisieren, Validierung, Fehlermaße	B, A
2	GeoSphere-Daten einlesen, Anwendungsfall rechnen	C
3	Sensitivitätsanalyse	C
3	Simulink-Modell und Batterie	D
4	Protokoll ausformulieren, Abbildungen final, Präsentation	alle

Phase 0 ist keine Formalie. Wenn die Systemgrenze und die Annahmen nicht gemeinsam festgelegt sind, schreibt jeder später ein anderes Modell auf.

Datenbeschaffung sofort. Alles ab Aufgabenpunkt 5 hängt an den GeoSphere-Daten. Erfahrungsgemäß hängt es dort: Stationsauswahl, Datenlücken, Zeitzone (UTC oder MEZ), und ob Globalstrahlung horizontal oder in Modulebene vorliegt. Diese vier Punkte klären, bevor jemand anfängt zu rechnen.

5 Ordnerstruktur

```
pv_projekt/
  matlab/
    init_parameters.m      alle Parameter, einzige Stelle mit Zahlen
    pv_thermal_ode.m      rechte Seite der Bilanz
    calc_w_el.m           elektrische Leistung
    calc_h_conv.m         Waermeuebergangskoeffizient
    calc_q_rad.m          Strahlungsverlust
    calc_errors.m         MAE, RMSE, MBE
    build_weather_struct.m baut die Interpolanten
    load_weather_paper.m   Validierungsdaten
    load_weather_geosphere.m Messdaten Anwendungsfall
    run_validation.m       rechnet -> results/validation.mat
    run_usecase.m          rechnet -> results/usecase.mat
    run_sensitivity.m      rechnet -> results/sensitivity.mat
    plot_validation.m      plottet <- results/validation.mat
    plot_usecase.m
    plot_sensitivity.m
    fig_style.m           einheitliches Aussehen
    save_figure.m         Export als Vektor-PDF
    run_all.m             Steuerung ueber Flags
    data/ results/ figures/ simulink/
  latex/
    main.tex
    kapitel/01_einleitung.tex ... A_anhang.tex
    literatur.bib
  doku/
    Projektvorgehen.pdf   dieses Dokument
    annahmen.md           Entscheidungslog
```

6 Arbeitsregeln für den Code

- R1. Keine Zahlenwerte außerhalb von `init_parameters.m`.** Wer eine Zahl direkt in eine Rechnung schreibt, muss sie später in drei Dateien suchen.
- R2. Rechnen und Plotten sind getrennt.** `run_*.m` speichert nach `results/`, `plot_*.m` lädt von dort. Wer nur die Achsenbeschriftung ändert, startet keine Simulation neu.
- R3. Alles in SI, Temperaturen intern in Kelvin.** Umrechnung nach Grad Celsius erst beim Plotten. Einheitenfehler sind die häufigste Fehlerquelle und die am schwersten zu findende.
- R4. Gleichungsnummern synchron halten.** Jede Gleichung im Protokoll bekommt ein `\label`, im Code steht darüber `% G1.` (2.3). Kostet fünf Minuten und beantwortet in der Prüfung jede „wo steht das im Code“-Frage sofort.
- R5. Sprechende Namen.** `T_module` statt `x1`, `h_conv` statt `hc`.
- R6. Eine Datei, ein Zweck.** Über etwa 100 Zeilen steckt meist eine Funktion drin, die ausgelagert gehört.
- R7. Keine anonymen Funktionen mit Logik.** Ein Wrapper der Form `@(t,T) pv_thermal_ode(t,T,p,w)` ist in Ordnung, alles darüber nicht.
- R8. Vektorisierung nur, wo sie spürbar etwas bringt.** Eine lesbare Schleife schlägt einen cleveren Einzeiler. Wir werden auf Nachvollziehbarkeit bewertet, nicht auf Rechenzeit.

R9. Kein Feature ohne Aufgabenbezug. Wenn niemand sagen kann, welchen der acht Punkte eine Funktion bedient, fliegt sie raus.

R10. Versionierung. Git, oder zumindest datierte Ordner. Nicht `final_final_v3.m`.

6.1 Zwei technische Stolpersteine

Wetterdaten müssen interpoliert werden. Der ODE-Solver wählt seine Zeitschritte selbst und fragt Zeitpunkte ab, die zwischen den Messwerten liegen. Die Interpolanten werden einmal in `build_weather_struct.m` erzeugt und in die Struct `w` gelegt, nicht bei jedem Aufruf der rechten Seite neu. Lineare Interpolation, weil ein Spline bei Bewölkungssprüngen überschwingen und negative Einstrahlung erzeugen kann.

Die Solverwahl muss begründet werden. Unser System ist skalar, ein Steifheitsverhältnis ist damit gar nicht definiert, ein steifes Problem kann es also nicht sein. Daraus folgt ein explizites Verfahren, konkret `ode45` (Dormand-Prince, Ordnung 5 mit eingebetteter Ordnung 4 zur Schrittwweitensteuerung). Toleranzen setzen wir bewusst auf 1×10^{-6} statt die Defaults zu nehmen, und wir zeigen einmal, dass sich das Ergebnis bei strengerer Toleranz nicht mehr ändert. Genau das ist der Punkt, an dem die Vorlesung geprüft wird, „haben wir immer so gemacht“ reicht nicht.

7 Arbeitsregeln für das Protokoll

Kapitelstruktur:

1. Einleitung und Aufgabenstellung
2. Modellbildung (Systemabgrenzung, Bilanz, Zustandswahl, Teilmodelle, **Annahmentabelle**)
3. Modellparameter (Recherche, Parametertabelle mit Quellen)
4. Numerische Umsetzung (Programmstruktur, Solverwahl, Interpolation)
5. Validierung (Vorgehen, Fehlermaße, Ergebnisse, Diskussion der Abweichungen)
6. Anwendungsfall (Datengrundlage, Ergebnisse, dominanter Verlustterm)
7. Sensitivitätsanalyse
8. Simulink und Batterieladung
9. Diskussion und Grenzen des Modells
10. Verwendung von KI-Werkzeugen

Jedes Kapitel ist in Overleaf eine eigene Datei unter `kapitel/`. So können vier Leute gleichzeitig schreiben, ohne sich zu blockieren. Ganz oben in jeder Datei steht ein Kommentarblock mit **VERANTWORTLICH:** und einer Liste, was in dieses Kapitel gehört und was nicht.

7.1 Die Annahmentabelle

Das ist das Stück, das über die Note entscheidet. Drei Spalten: was wurde angenommen, warum ist das zulässig, was kostet es voraussichtlich. Beispiel:

Annahme	Begründung	Erwartete Auswirkung
Modul als ein Temperaturknoten	Laminatdicke rund 3 mm, Biot-Zahl klein	kein Gradient abbildbar, im Mittel vernachlässigbar
Bodentemperatur gleich T_{amb}	keine Messdaten verfügbar	unterschätzt Rückseitenverluste bei aufgeheiztem Untergrund

Regel: neue Annahme im Code heisst neue Zeile in der Tabelle. Wer im Code etwas vereinfacht, ohne es einzutragen, hat die Arbeit nur verschoben, nicht erledigt.

Parallel dazu führen wir `doku/annahmen.md` als Entscheidungslog: Datum, Entscheidung, Begründung, wer. Eine Zeile pro Eintrag, keine Prosa. Daraus wird später die Tabelle.

7.2 Abbildungen

Die Protokolltipps der LVA zeigen konkrete Negativbeispiele. Was dort kritisiert wird:

- fehlende Achsenbeschriftung, unbekannte Einheiten
- Legende vorhanden, aber unsauber beschriftet (etwa `phi0`)
- gleiche Linienart für alle Kurven, Unterscheidung nur über Farbe
- zu kleine Plots, Schriftgröße passt nicht zum Fließtext
- Redundanz zwischen Plottitel, Achsenbeschriftung und Bildunterschrift
- Simulink-Scopes direkt ins Protokoll kopiert
- nichtssagende Bildunterschriften wie „Simulationsgraph Simulation 2“

Die zugehörige Checkliste für wissenschaftliche Arbeiten verlangt zusätzlich: Vektorgrafik, sinnvoller Bildausschnitt ohne große leere Flächen, kein Informationsverlust im Schwarzweißdruck, fortlaufende Nummerierung, und eine Bildunterschrift, die die Abbildung unabhängig vom Text verständlich macht und die Kernaussage benennt.

Praktisch heißt das für uns: `fig_style.m` am Anfang jedes Plotskripts aufrufen, Export über `save_figure.m` als Vektor-PDF, unterschiedliche Linienarten statt nur Farben, und meist gar keinen Plottitel setzen, weil die Bildunterschrift das übernimmt.

7.3 KI-Werkzeuge

Die Nutzung ist erlaubt und muss nicht verschleiert werden. Kapitel 10 beschreibt: welche Werkzeuge, wofür konkret, und vor allem *wie geprüft wurde*. Der letzte Punkt ist der, auf den es ankommt. Jede übernommene Formel wird gegen die Primärquelle geprüft, jeder Parameter gegen die zitierte Arbeit. Was verworfen wurde, wird ebenfalls erwähnt.

8 Offene Punkte

Was	Wer	Bis wann
GeoSphere-Datensatz auswählen und herunterladen	C	
Zeitzone und Datenlücken klären	C	
Paperdaten aus Tuncel Abb. 1 digitalisieren	B	
Umgebungstemperatur für die Validierung festlegen (Paper macht keine Angabe)	B	
Alle TODO Quelle in <code>init_parameters.m</code> belegen	A	
Schichtaufbau für C_m mit Quellen hinterlegen	A	
Modulausrichtung und Neigung festlegen	alle	
Batteriemodell von der LVA anfordern	D	
Simulink-Onramp durcharbeiten	D	

9 Woran wir merken, dass es gut läuft

- Jede Zahl im Code hat eine Quelle im Kommentar.
- Jede Gleichung im Protokoll hat eine Nummer, die im Code vorkommt.
- Die Annahmentabelle wächst mit dem Code mit, nicht erst am Schluss.
- Wer ein Kapitel nicht geschrieben hat, kann es trotzdem erklären.
- Ein `plot_*.m`-Aufruf braucht keine Neuberechnung.